Workshop starts at 12:03

The Researcher’s Guide to Code Organization

by efrén cruz cortés

TA: Yangyang Li

https://github.com/nuitrcs/researchers_guide_to_code_organization

This workshop is brought to you by:

Northwestern IT
Research Computing and Data Services

Need help?

- AI, Machine Learning, Data Science
- Statistics
- Visualization
- Scraping, text analysis, modeling, computing, reproducibility, etc.
- R, Python, SQL, MATLAB, Stata, SPSS, SAS, etc.

Request a **FREE** consultation at **bit.ly/rcdsconsult**.

Logistics

- Ask questions!
  - If I use terminology and forgot to define, please ask!

- Feel free to interrupt and ask whatever for clarification

Downloading the materials

https://github.com/nuitrcs/researchers_guide_to_code_organization

What to expect

- A lay of the land
  - won’t be able to go into details
- Everything you always wanted to know about programming for research but were afraid to ask
- Hands on exercises (mostly Day 2)

Overview

- Day 1 will focus on overall project organization principles and tools.
- Day 2 dives into organizing principles within a script.

Day 1 Introduction

Does this sound familiar?

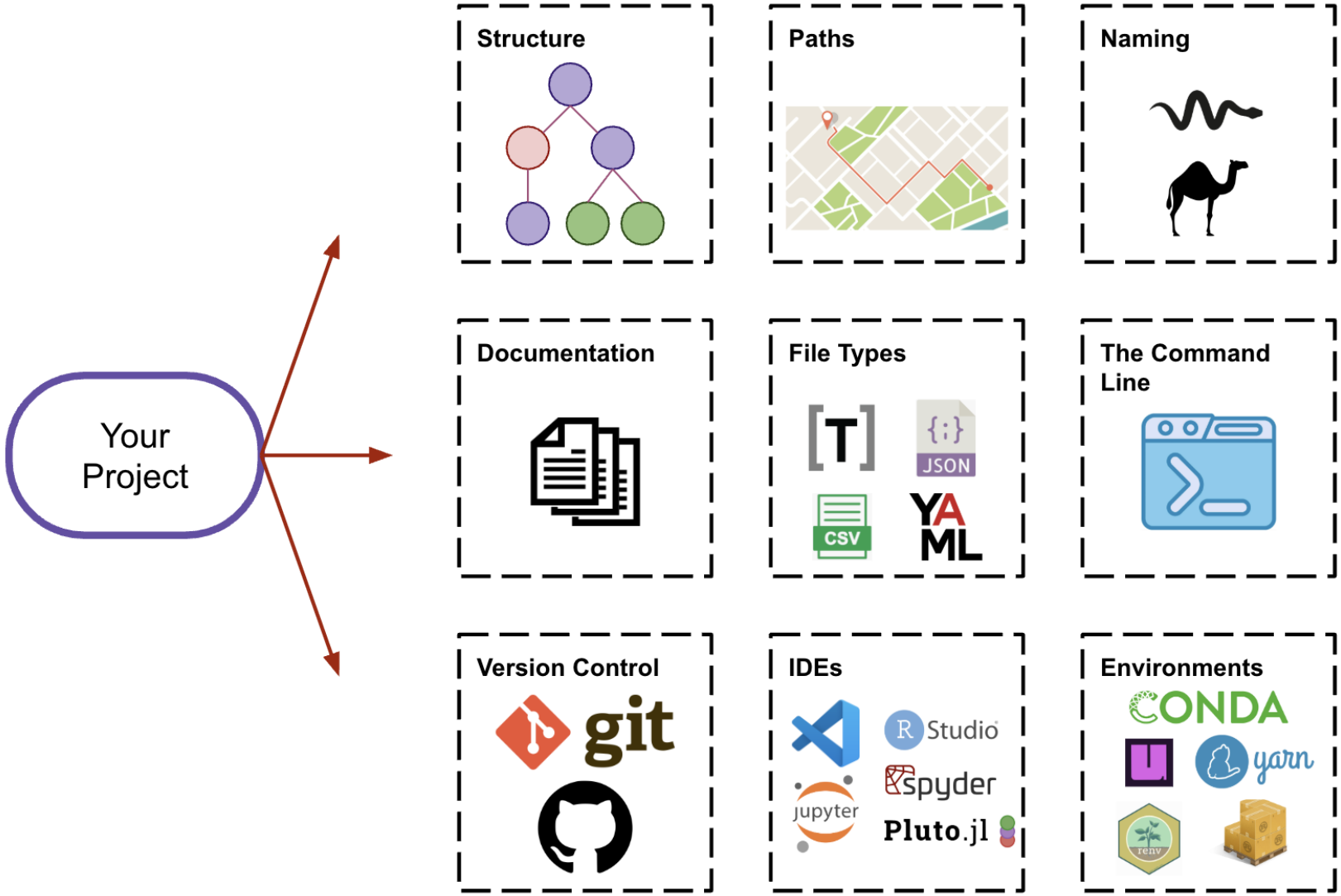
- I can’t find my script!
- My script worked last week but now it’s broken.
- My script works perfectly in my computer but not when I share it with my colleague.
- What does this file do? I’ll just delete it…

Motivation

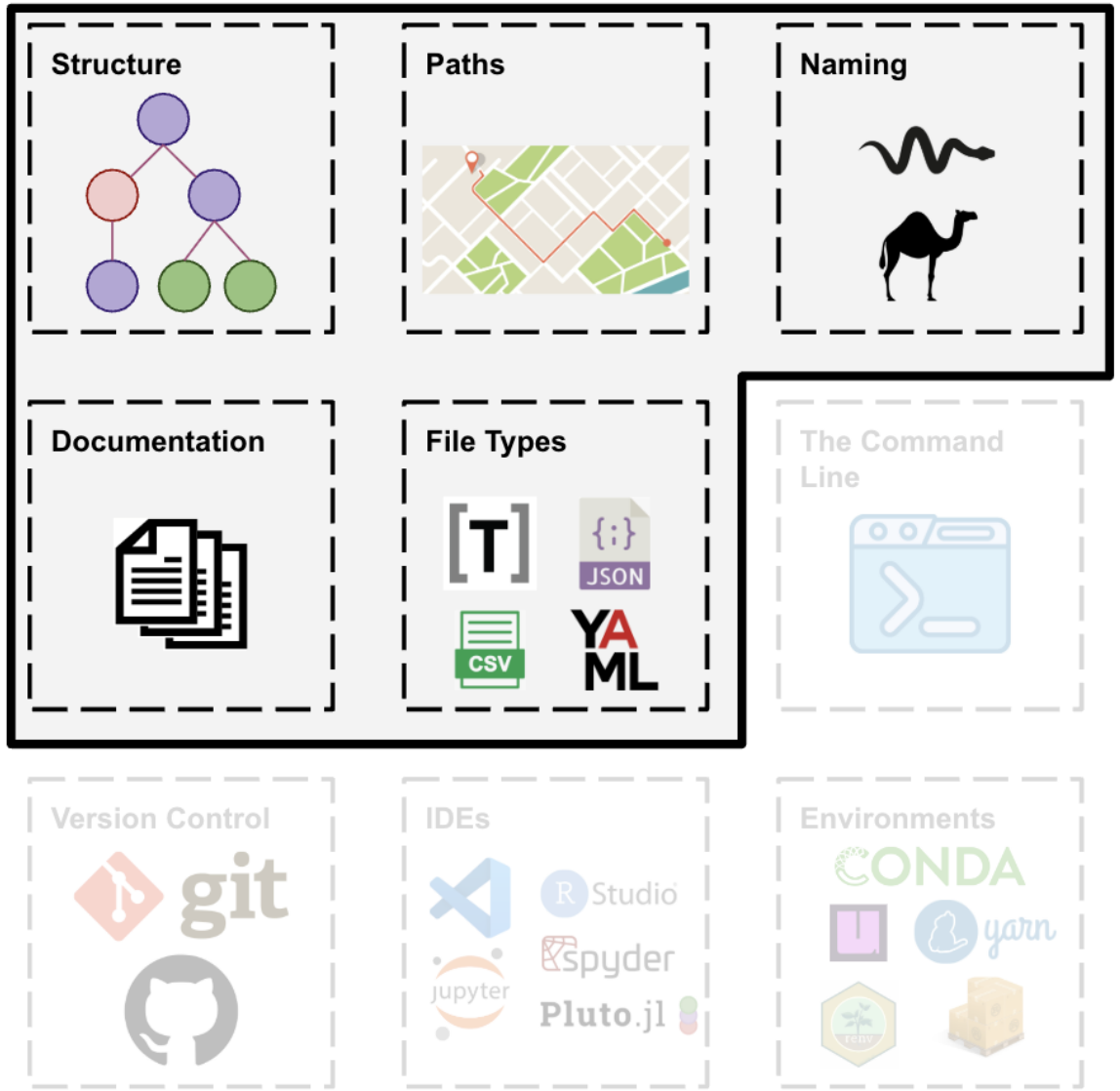
- A code-based research project is composed of many elements outside internal code logic, and yet, these are not taught in classes!
- We need a place to understand how all these elements come together, why they are necessary, and the principles behind them.
- Proper organization will facilitate collaboration, reproducibility, and other important elements of research.

A bit of effort now will save you sweat and tears in the future!!

Day 1- Overview



Internal Structure



Structure

Paths

Naming

Documentation

File Types

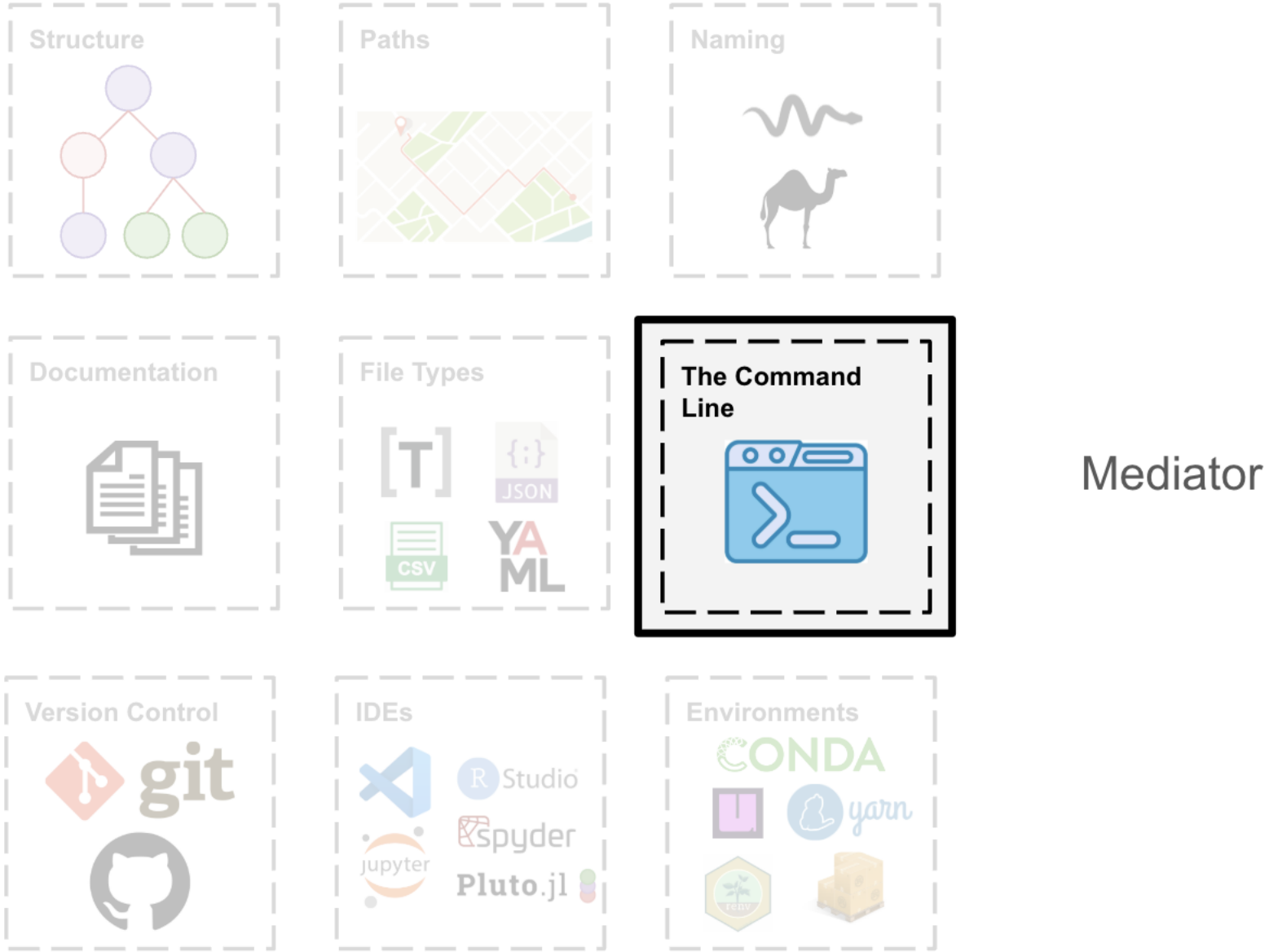
The Command Line

External Tools

Version Control

IDEs

Environments


Structure
Paths
Naming
Documentation
File Types
JSON
CSV
YAML
The Command Line
Version Control
git
IDEs
Studio
spyder
jupyter
Pluto.jl
Environments
CONDA
yarn

Mediator

General Principles

- Organization
  - Separation of Concerns
  - Modularity
  - Containment
  - Hierarchy
- Robustness
  - Usability
  - Portability
  - Automation
  - Scalability
  - Reliability
- Collaboration
  - Documentation
  - Transparency
  - Reproducibility
  - Version Control

General Principles

- Organization
  - Separation of Concerns
  - Modularity
  - Containment
  - Hierarchy
- Robustness
  - Usability
  - Portability
  - Automation
  - Scalability
  - Reliability
- Collaboration
  - Documentation
  - Transparency
  - Reproducibility
  - Version Control

Architecture

Performance

Continuity

Quick Survey

- What is your primary programming language?

- Do you use an integrated development environment (IDE)?
  - (e.g., RStudio, Jupyter Lab, VS Code, ...)

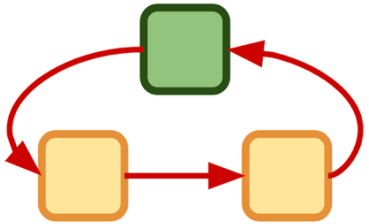
Project Structure

Identify your project structure

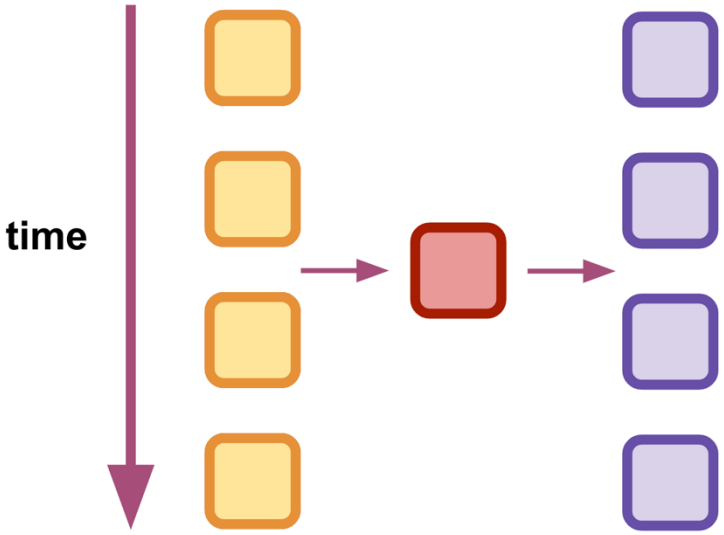
- Does your project use data?
- If so, at which stage?
  - At the beginning (training, preprocessing, …)
  - At the end (calibration, validation, …)
  - Throughout (sensors, inference, …)
- Is your project iterative in nature?
- Does your project branch into possible models / analyses?

A simple project taxonomy*

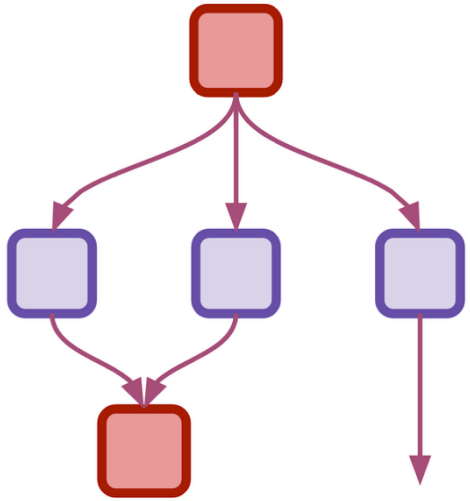
Linear



Iterative



Recurrent



Branching

*I couldn't find an official taxonomy / framework for research project structures, so these examples are limited to my own experience. -ecc

Examples

In reality, most projects are iterative in nature as we improve our own understanding and as we calibrate with data.

These are generic possibilities, but of course research is more complex:

- **Linear:** batch learning, descriptive statistics
- **Recurrent**: online learning, sensor data
- **Iterative**: physics simulation, agent-based models
- **Branching:** model selection, by-case analysis

Exercise

Grab a piece of paper and sketch the structure of your project.

- It's OK if it isn't perfect, if it doesn't fit the shapes above, or if it is a strange mix.
- Show it to **your neighbor** and talk about it!

Directory Structure

Question

- What is the first thing you do when you start a project?
- When you create a new script, where do you save it?

Keep the project contained

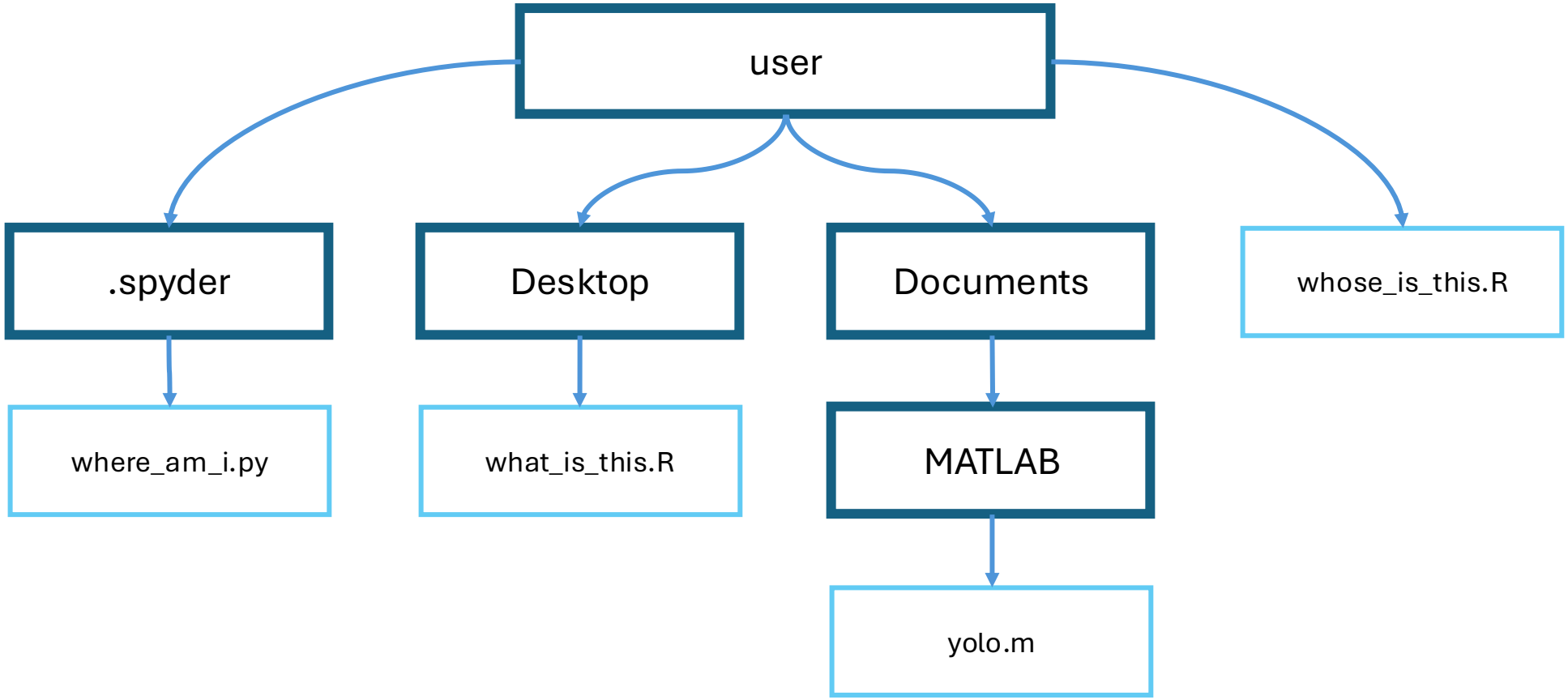
By default, all materials for your project should go inside one **project directory**.

(Imagine the chaos of saving all your .doc files in a single folder)

Plan your project directory based on your project structure.

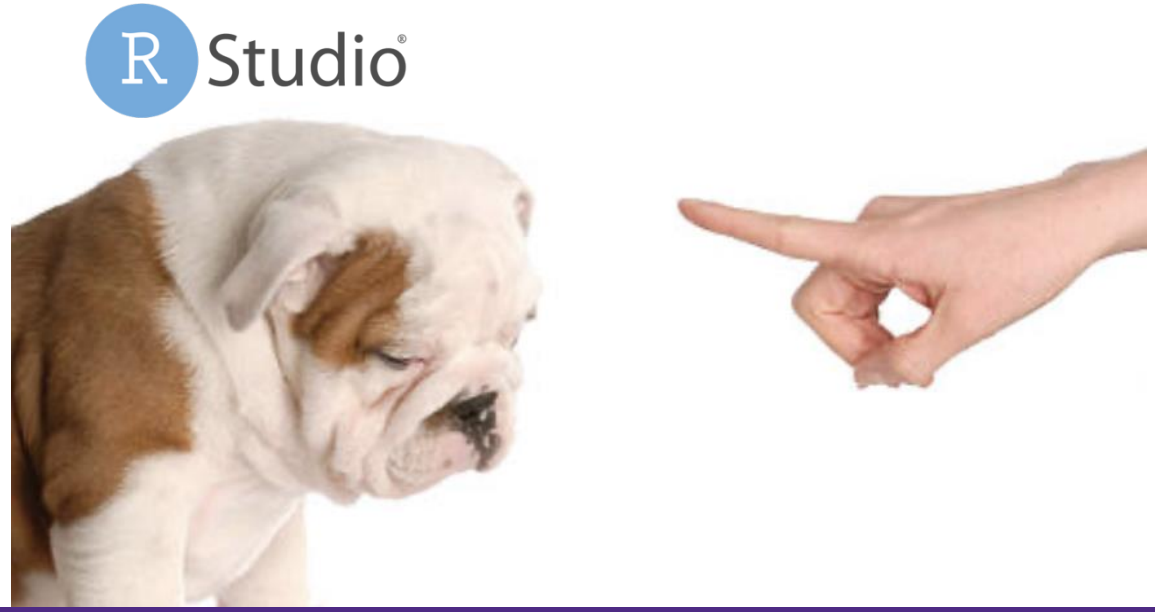
Bad!

See example folder ***Example-Bad***



Bad IDEs!

- IDEs by default tend to save your files in predetermined folders (often hidden) they create during installation.
- You should NOT save your files in your IDE's default, but within your project directory.



Remember

Create a project directory, and keep your files contained within it.

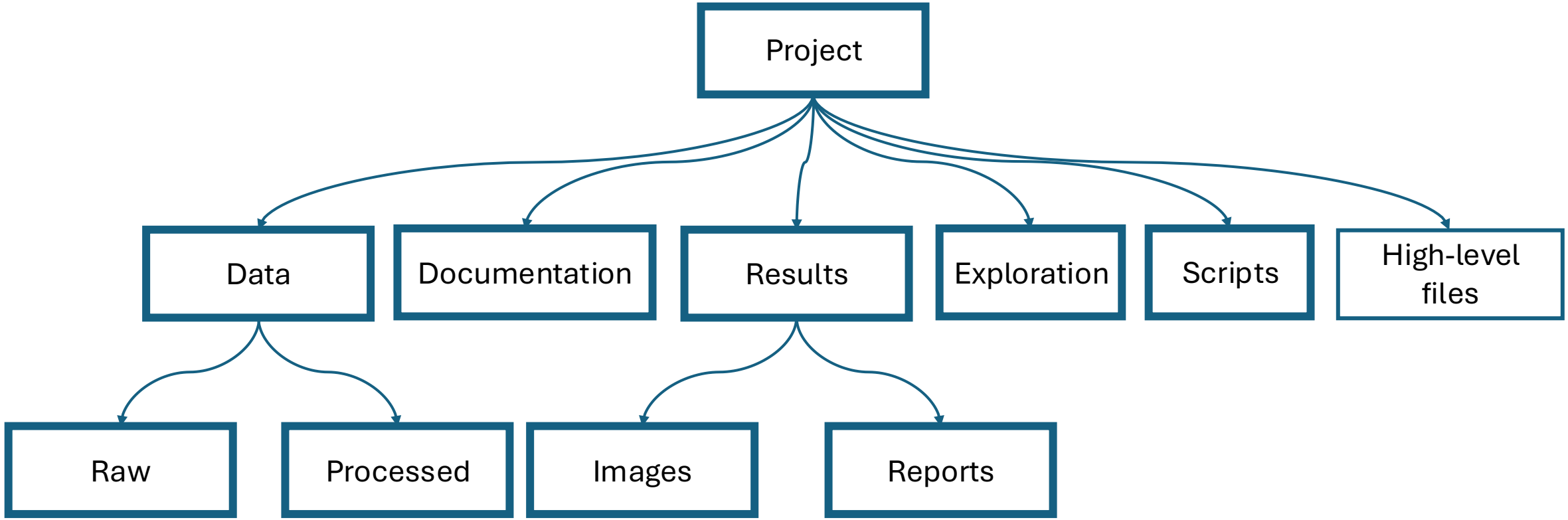
Separation of Concerns

Separation of concerns is the general principle in which pieces of a project should be separated according to their distinct form and function.

For example, data is separated from “code” files.

Within a research pipeline, preprocessing is separate from analysis, which is separate from visualization.

Good! Common organization paradigm



Example

See example folder ***Example_Desktop/Project_1***

No size fits all

The previous organizational paradigm is common and a good starting point, but your project may ask for a different structure.

Just make sure you abide by general principles of containment and organization.

Tree structures

Note that you won't always be able to visualize a tree as above. Sometimes the tree structure is written out using tabs and/or lines:

```
Project
| - Data
    | - Raw
    | - Processed
| - Scripts
| - Results
    | - Figures
    | - Reports
| - high_level_file
```

Exercise

Grab a piece of paper and create a draft of your directory structure based on the first exercise.

- Think about where you will save files by function and by type.
- You can draw a tree diagram or write it out in linear format as above.

Then, **create the respective directories in your computer**!

Note on literature

You may be tempted to add supporting literature (research papers, etc.) to your project directory. However, you will often use overlapping literature for different projects.

Instead, I recommend you organize literature elsewhere and use a reference management tool like **Zotero**.

General Principles

- Containment
- Separation of concerns
- Modularity
- Hierarchy
- Efficiency

Intermezzo 1 - Paths

The current working directory

- A directory is a location in your computer (a folder in the desktop, for example)

- When running a script, the *current working directory* (cwd) is the location from where the script is run (set by IDE or terminal)

- Not all scripts are at the same location as the cwd, but we should aim for them to be “close by” (see next slide).

Absolute vs relative paths

- The *absolute* path of a file is the “address” from the root directory of your computer.
  - /Users/efren/Desktop/Example/some_file.txt

- A *relative* path is the address as viewed from a different starting point. For example, if I’m on the Desktop already:
  - Example/some_file.txt

Where’s your Desktop?

Note:

On some new laptops, your Desktop folder may be automatically in either iCloud or OneDrive.

- This means your files are not saved in your computer but in the cloud!
- Just make sure you know where everything is, and are intentionally using one option or the other.

Project’s Root

- The top level directory of your project
- From there, all subdirectories branch off, as in a tree
- Tomorrow we’ll talk more about the project root

Don’t mistake “project root” for just “root”, which usually refers to the top level directory of your whole computer.

Use relative paths!

Common relative path indicators:

- Single dot (.) indicates current directory
- Double dots (..) indicate parent directory
- ‘A/B’ indicates B is a sub-directory of A
  - Data/Raw (Mac)
  - Data\Raw (Windows)
- Lone slash ‘/’ for mac, or ‘C:’ for windows at the beginning mean computer’s root directory
  - /Users/Alice/Desktop/Project_1 (Mac)
  - C:\Users\Alice\Desktop\Project_1 (Windows)

Assume I’m on the desktop:

./Project_1/Data

../Downloads

Path Standardization

Paths are written differently for different operating systems (forward and backward slashes being the main culprits)

- Many languages have a tool to help you standardize paths.
- In python, for example, `pathlib` does that for you.

Example

See jupyter notebook inside folder ***Example_Paths/Project_Root***

General Principles

- **Portability**
  - You can move your folder around and it won’t break.
- **Collaboration**
  - You can share your project with other people!

Exercise

Assume you are situated in “Project_1”. Change the following absolute paths to relative:

- /Users/you/Desktop/Project_1/data/your_data.csv
- /Users/you/Desktop/Project_1/auxiliary_file.py
- /Users/you/Desktop/Project_2/
- /Users/you/Downloads

If you are stuck talk to your neighbor!

Naming

PROTIP: NEVER LOOK IN SOMEONE ELSE'S DOCUMENTS FOLDER.

Conventions

- Be **specific** and be **descriptive**
- Be **consistent**
- Name according to specific function, stage, date, etc.
- Avoid generic names like *data.csv*, *function.py, etc.*

Conventions – versioning

- Avoid file names with version suffixes like:
  - *my_file_final.R*
  - *my_file_final_final.R*
  - *my_file_final_final_FOR_REALZ_THIS_TIME.R*
- Try to avoid manual versioning (we'll talk version control soon)
  - *my_file_v1.m*
  - *my_file_v2.m*
- For data, dating is better (see next slide)

Conventions - dating

- Use the standard date format
  - YYYY-MM-DD
  - YYYY_MM_DD
- Most languages have a function to obtain current date/time.
- Get in the habit of appending date to data files names as either prefix or suffix.
- When necessary, also append time of simulation (hour, minute, second).

Conventions – casing for legibility

- Recommended:
  - snake_case: *my_file.py*
  - camelCase: myFile.js
  - mixed: *heartData_Evanston_2025_02_24.csv*

- Acceptable:
  - hyphenated-case (usually for folders): *project-1*

- Avoid:
  - spaced names: *project report.pdf*
  - dotted*: *heat.data.evanston.csv*

* Some authors recommend dotted naming. I discourage it as it is quite confusing and unacceptable under some language conventions.

A Note on Data, Dates, and Names

Occasionally, I generate a lot of data files within a single simulation. In this case, instead of naming:

- YYYY_MM_DD_hh_mm_ss_simulation_001.mat
- YYYY_MM_DD_hh_mm_ss_simulation_002.mat
- YYYY_MM_DD_hh_mm_ss_simulation_003.mat

I make a folder *per run* inside my results folder, and save only the generated data of that run in there:

- YYYY_MM_DD_hh_mm_ss/
  - simulation_001.mat
  - simulation_002.mat
  - simulation_003.mat
  - simulation_info.txt

General Principles

- Usability
- Portability
- Automation
- Collaboration

Documentation

Where to document

- A README file for the whole project.
- Specific observations / thoughts, etc. (appropriately placed in directory tree structure).
- Inside your script.

README file

- Overall description of the project
- General requirements
- Other general information (structure, explanations, semantics, conventions)
- Example usage

Example

See example ***Example_Desktop/Project_2***

General Principles

- Reproducibility
- Collaboration

txt vs md

My README file had a weird suffix: **.md**

What is this about? Well...

Intermezzo 2 – File Types

What should README be?

- You may have seen README files as:
  - *README.txt*
  - *README.md*
- The latter one is a markdown file
  - This just means it has extra marks for nice formatting, used often in the web.

Different data types

- Do not be afraid of different file types, you will encounter and work with many, and it’s fine.
- Learn what each does and how it interacts with others in your project

Data: language specific vs agnostic

- You may be used to saving your data in language-specific formats:
  - .pickle, .pkl, .mat, .rds, .rdata, etc.
- These formats are fast and convenient (often binary), but are not portable and easily shareable.
  - If your data is huge, you may be forced to use them.
- If you want to share data, aim for language-agnostic formats, at least for finalized data:
  - .csv, .json
- If you don't know what a file type is, look it up!

Auxiliary files

You will encounter, and use, a variety of auxiliary files:

- To store directory information, configuration parameters, “secrets” (API keys, e.g.), requirements, version control info, etc.

- As metadata: to help you read *other* data files.

Example

EEG Data (BrainVision format)

- .eeg → raw, binary data of voltage measurements
- .vhdr → header file, metadata like sampling rate
- .vmrk → records specific times of triggers or events

Your data is not always in one single file!

Common Project files

We'll talk about these more in Day 2, but pay attention:

- .env (usually for secrets and other environment variables)
- .yaml, .toml (usually for configuration parameters, requirements)
- .gitignore (which files to ignore by Git, more on this later)

Seeing hidden files

- Invisible files start with a single dot “.”
  - .gitignore
  - .env

- You can see them in your computer, while on your file explorer:
  - Mac: command + shift + .
  - Windows: File Explorer -> View -> Hidden items

Creating files

- You will need the command line or and IDE to create and edit those files.

- More on IDEs later.

Exercise

Experiment with the IDE you are used to (Jupyter Lab, RStudio, etc.) and see what type of files you can create and edit.

Make a mock README.md file in your project directory.

General Principles

- Portability
- Reproducibility
- Collaboration

The Command Line

The command line interface (CLI)

- Text based interface to communicate with your operating system
  - Terminal
  - Console
  - Prompt

- You will certainly need it for your projects, but don’t be afraid, you don’t need to learn everything about it.

Analogy

When working on a powerpoint presentation, there are “add-ons” or “plug-ins” that help you achieve X, but are not default implementations of powerpoint.

Similarly, programming projects often rely on external tools for specific purposes (downloading packages, version control, connecting to remote servers, etc.)

These tools don’t come with a nice graphical interface (too much work!), but you can run them on the command line!

- their use can be automated
- commands are precise and reproducible

Who’s afraid of the CLI?

- Many tools will require you to run commands from the CLI, if anything just for installation.
- Usually, these tools have clear instructions on how to use it and what to run.
- You don’t need to learn everything now. Important steps:
  - Get comfortable with opening and closing it
  - Learn how to change directories

Example

- Changing directories
- Listing directories

- See live example

```
# cd stands for change directory. what follows is
where to, relative to current
cd ../
```

```
# ls (or dir in windows) lists the items in the
current directory
ls
```

Exercise

Open your:

- PowerShell if on Windows
- Terminal if on Mac

1. Look up the commands to change directory and to list files online
2. Play around for a bit!

Version Control

What and why?

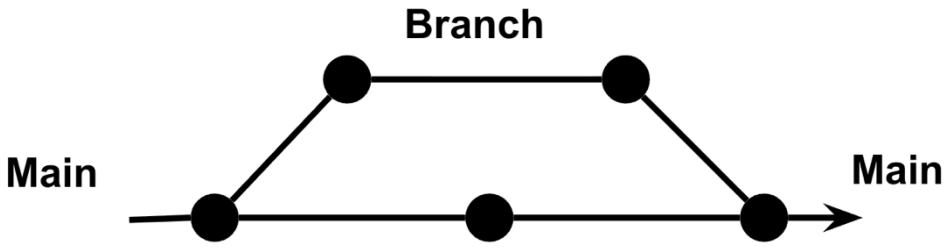
- Version control is any protocol used to keep track of versions of your code (or other materials) as you develop it.
- A version control system is any specific software that helps you achieve the above.
  - Helps you avoid manual version control: *file_v1*, *file_v2*, etc.
- The most common vcs is Git
- Using a version control tool is indispensable for efficient and robust collaboration!

Example

- Each point is a new version of your code. You can keep track of these on the central line.

- You may want to experiment with changing some aspects of your code, so you branch off, and later "merge" with the main line.

- Alternatively, you may share with a collaborator, who will make changes, and then merge.



Git

- We don't have time to help you install and learn git, but we have materials here.
- It's easy to use, and to get started you need to learn 5 words:
  - init, status, add, commit –m

- (Live demonstration)

GitHub

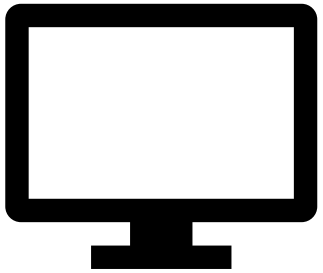
- GitHub is a website that allows you to:
  - Host Git repositories online
  - Perform Git actions through a graphical interface

- You can sync your local project with GitHub in several ways:
  - Through a command line interface (terminal, etc.)
  - Through an IDE
  - Through the GitHub Desktop App

- I recommend starting with the terminal, may take a bit of searching and learning but the payoff is worth it

Example

lab computer

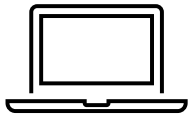


solver

Quest

solver_alice

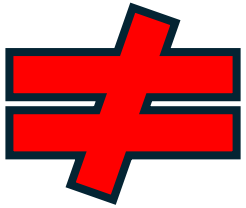
solver_alice



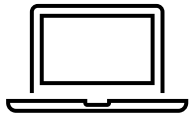
Laptop

Quest

solver_bob

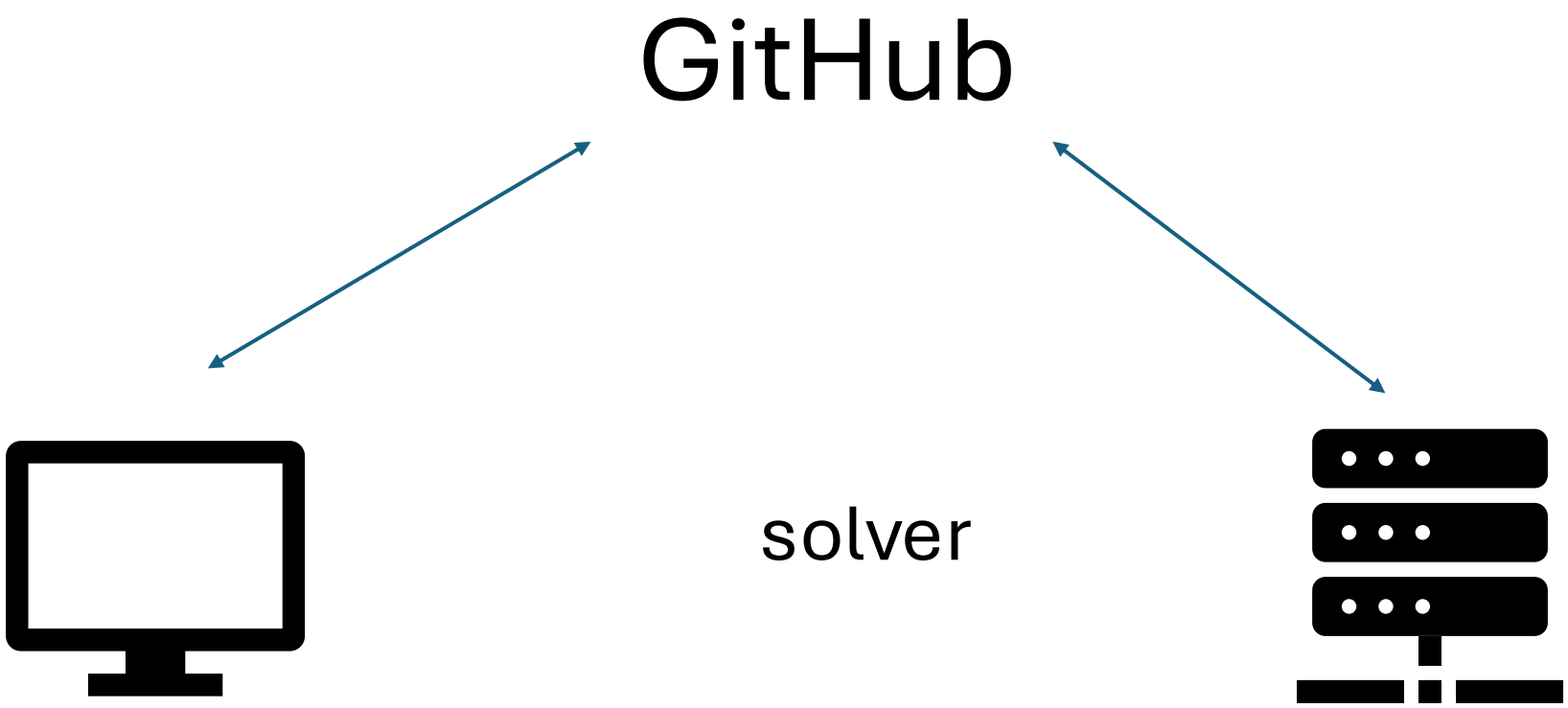


solver_bob



Laptop

Example



.gitignore

VERY IMPORTANT

The *.gitignore* file tells Git which files to ignore. This is a very important file if you use GitHub or another public repository to avoid sharing **secret information** (for example API keys) online.

You can find *.gitignore* templates for your favorite programming language here.

Never upload secret or sensitive information to GitHub!!

General Principles

- Collaboration
- Version Control
- Reproducibility

Integrated Development Environment

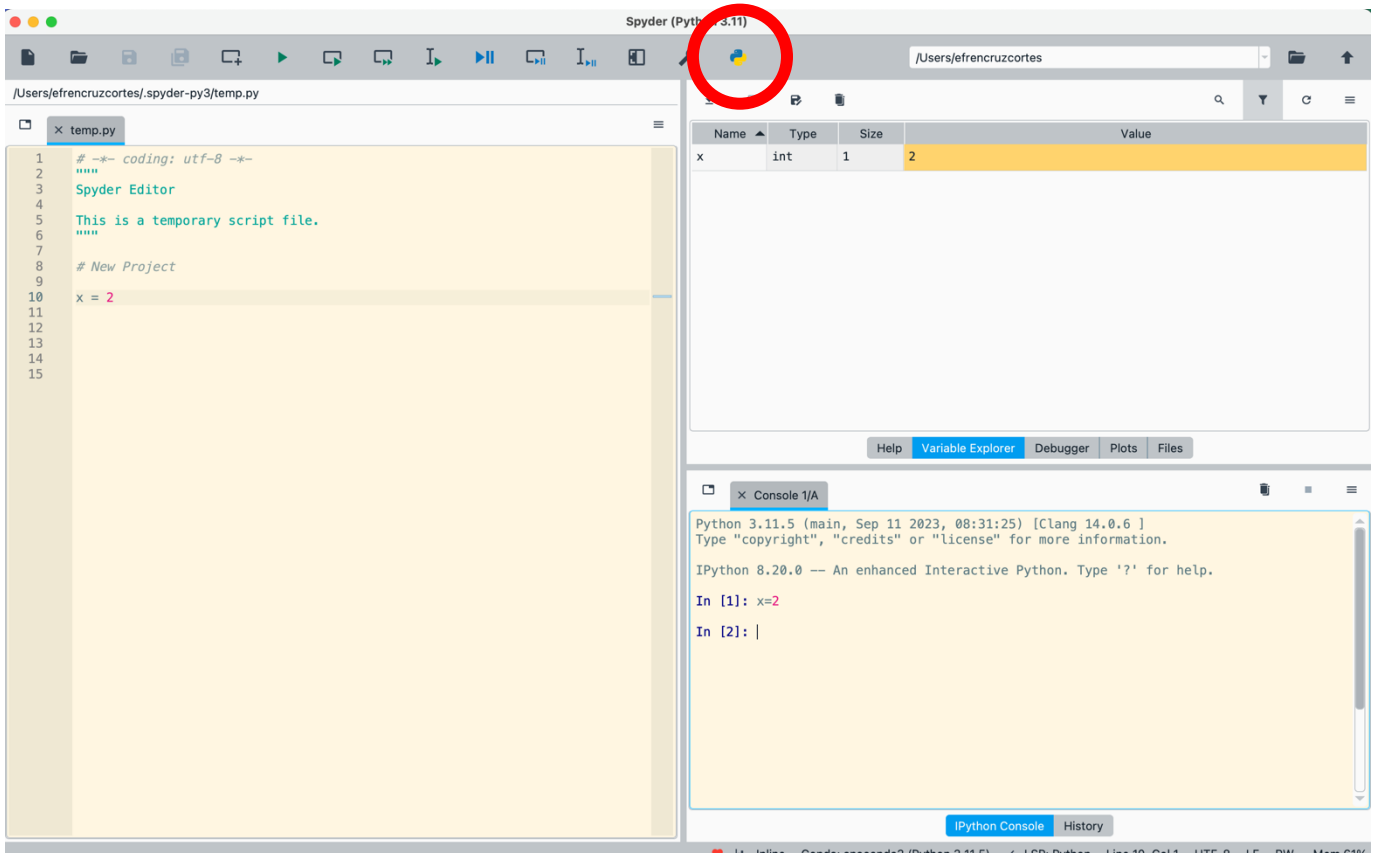
IDEs

Integrated Development Environments (IDEs) have many advantages besides editing scripts

- Enhance productivity
- Facilitate project management and Organization
- Help with version control
- Help with debugging

IDEs

IDEs have tools to help you manage projects. For example path management, “projects” (R and Spyder), etc.



Examples

- Visual Studio Code
  - Most versatile
  - Supports many languages
  - Supports many file formats
  - Takes a bit of extra time to get used to
- Jupyter Lab
  - Ju-py-(te)r: supports Julia, Python, R
- RStudio
  - R - Best for R and de-facto.
- Others:
  - Spyder – For Python. Similar GUI to RStudio.
  - Pluto – For Julia. Interactive like Jupyter, but nicer.
  - Matlab's own.

Choose your own adventure – but choose well!

- If your language is proprietary (like MATLAB), you don't have a choice.
- If you are only working in R → RStudio
- New-ish to python:
  - Spyder if using *.py* scripts, want to explore variables, variable structure, easily peak into class methods and properties, etc. Spyder "projects"
  - Jupyter if using *.ipynb* notebooks, are doing exploration, or following/preparing educational materials.
- More experienced, multi-language, and a bit of patience:
  - Visual Studio Code.
  - I'm a convert to VS Code. It is fast, helps you easily create auxiliary files of all types, supports many languages, etc.

Aside note on Jupyter notebooks

- Jupyter notebooks are popular because of their pedagogical value: it is easy to present step by step procedures.
- If you use them, use them for pedagogical reasons or for exploration and experimentation.
  - Make sure you add a folder akin to “exploration notebooks” to your directory tree!
- But, you should aim to have full *.py* scripts as your final result.
  - Containment.
  - Consistency
  - Reproducibility.
  - Automation.

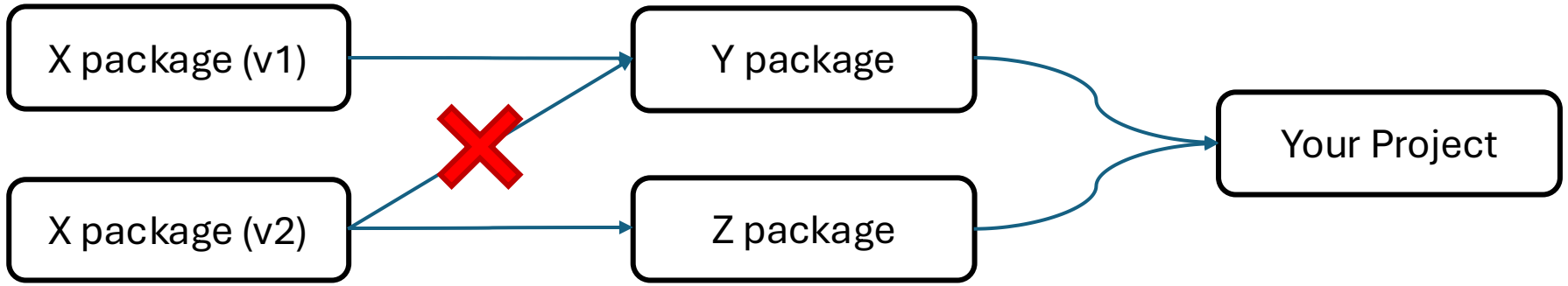
Environments

Environments

- An environment is an isolated space, generally self-contained, containing dependencies necessary to run a project.
- Dependencies include libraries, packages, and modules.
  - These are just pre-written pieces of code
- Environments contain specific versions of the project's dependencies.
- They avoid conflict among different dependencies and their versions.

Dependency hell

- You can't build everything from scratch. You will inevitably depend on libraries built by others.
- These are built mostly independently, so they can conflict with each other.
- When dependencies are in conflict with each other, your project may encounter unexpected errors and become impossible to run.



The proprietary paradise

If you are a MATLAB user (or use another proprietary language) you may not be used to this concept, since most packages are designed by the company and kept consistent to avoid these conflicts.

Environments and package managers

- Each language will have a tool to create environments, as well as to manage packages within an environment.
- These packages are often downloaded from a repository.
- Packages, repositories, and managers have varying degrees of security and security checks, so be careful when installing.

Environment cheat sheet*

Language	Terminology	Environment Creation Tool	Package Repository	Package Manager	Requirements Files
Python	• Virtual environments • Conda environments	• venv • conda	• PyPI • Conda	• pip • conda, mamba	• requirements.txt • environment.yaml
R	• Project environments	• renv • (.RProj)	• CRAN	• renv • install.packages()	• renv.lock
JavaScript	• node_modules	• npm • yarn	• npm registry	• npm • yarn	• node_modules • .pnp.cjs
Julia	• Project environments	• Pkg	• Julia general registry	• Pkg	• Project.toml • Manifest.toml
Rust	• Cargo projects	• cargo	• crates.io	• cargo	• Cargo.toml • Cargo.lock
MATLAB	• N/A	• Path management • Add-on manager	• MATLAB central file exchange	• Add-on Manager	• N/A

* I'd love to improve on this cheat sheet, if you have suggestions let me know! -efrén

Environments

- You should not be afraid of creating environments!
- I like to create one per project

Approaches

- One environment you can use from anywhere
  - conda, mamba, etc.
- One environment per project
  - uv, renv, etc.

Example - Conda

```
conda env list

conda create --name <environment_name>

conda activate <environment_name>

conda install <pckg_1> <pckg_2>

conda deactivate

conda env remove --name <environment_name>
```

Example - renv

```r
# Works best if combined with a .Rproj

renv::status()

renv::init()

renv::activate()

renv::install("pckg_1")
renv::install("pckg_2")

renv::snapshot() # updates renv.lock file!

renv::dependencies()

renv::deactivate()
```

Note

While we're mentioning environments at the end…

You should set yours up early on! Even before writing any script.

General principles

- Reproducibility
- Consistency
- Collaboration

Final Exercise

Exercise

Take a few minutes to study the structure of one of your current projects.

- Write some notes on how you would like to re-organize it based on what we learned today
- DO NOT MAKE CHANGES
- Talk to **your neighbor** about it (as long as it's not a secret project).

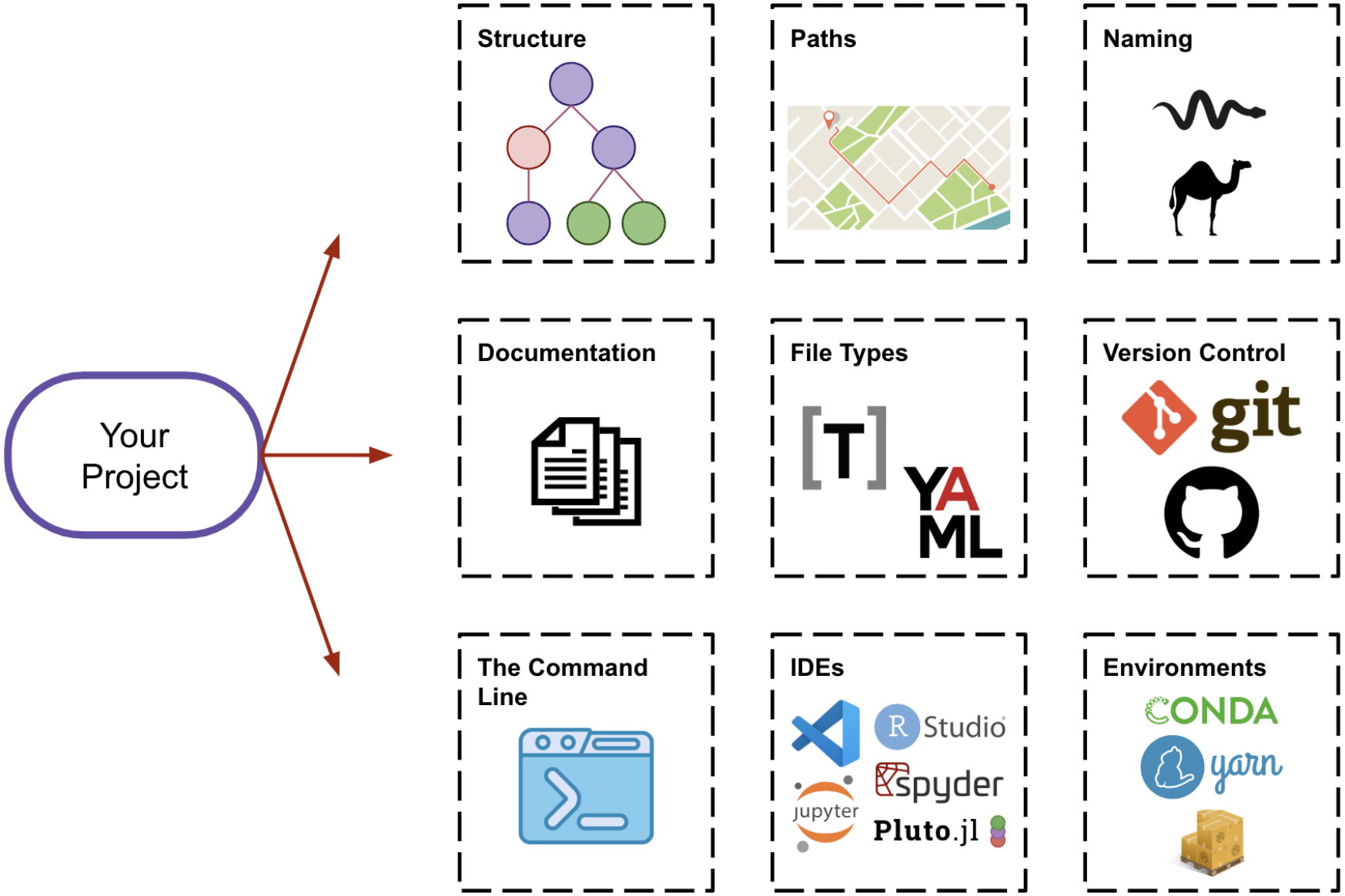
Chaos

Go to the Chaos folder, reorganize it and rename files based on the principles we learned today.

- All files in this example folder pertain to the same project.
- The mock files are empty, they are there just for practice.
- If there are folders or files that you think must be added, feel free to do so!

Recap

Day 1- Overview



Checklist

- Draft the structure of your project
- Create the initial directories
- Setup version control (git, GitHub)
- Setup an environment (renv, conda, etc.)
- Setup a README file
- Create and update configuration files in tandem to your coding
- Start coding!

Questions?

Other resources

This cool post on file structure:

- https://mitcommlab.mit.edu/broad/commkit/file-structure/

This workshop by our very own data management team:

- Tobin_Magle_Data_Management_Slides

Related workshops by our data science team:

- The command line
- Conda environments
- Git and GitHub